

Доклад на тему: Образование планетных систем

Дисциплина: Математическое моделирование

Оширова Ю.Н., Пронякова О.М., Сидорова Н.А., Тимофеева Е.Н.

Содержание

0.1	Этап 3. Реализация алгоритмов моделирования на Python	4
0.1.1	Введение	4
0.1.2	Цели	4
0.2	1. Алгоритм гравитационного N-тел	5
0.2.1	Описание	5
0.2.2	Пояснение к реализации	5
0.2.3	Код	6
0.2.4	1. Анализ результата: Алгоритм гравитационного N-тел . . .	8
0.2.5	Описание	8
0.2.6	Код	9
0.2.7	2. Анализ результата: Алгоритм гидродинамики вращающе- гося диска	10
0.2.8	Описание	11
0.2.9	Код	12
0.2.10	Что происходит на графике:	13
0.2.11	Как работает алгоритм:	14
0.2.12	Физическая интерпретация:	14
0.3	Интересные факты	15

Список иллюстраций

1	Код	6
2	Визуализация	7
3	График	7
4	Код	9
5	График	10
6	Код	12
7	График	13

Список таблиц

0.1 Этап 3. Реализация алгоритмов моделирования на Python

0.1.1 Введение

Формирование планетных систем — один из фундаментальных вопросов астрофизики. Современные численные методы позволяют исследовать эволюцию протопланетных дисков, поведение частиц под действием гравитации и процессов аккреции. На предыдущем этапе мы математически описали основные процессы. Сейчас переходим к практической реализации моделей на языке Python.

0.1.2 Цели

- Реализовать три модели:
 1. Гравитационное взаимодействие N тел
 2. Гидродинамику вращающегося диска
 3. Слияние частиц
 - Подготовить простой, понятный и расширяемый код
 - Продемонстрировать визуализацию процессов (по возможности)
 - Получить базовые численные результаты
-

0.2 1. Алгоритм гравитационного N-тел

0.2.1 Описание

Модель рассчитывает гравитационное взаимодействие между множеством тел. Уравнение движения каждого тела рассчитывается по второму закону Ньютона, сила — по закону всемирного тяготения.

0.2.2 Пояснение к реализации

- Используем численное интегрирование (метод Эйлера или Верле).
- Каждое тело имеет массу, положение и скорость.
- Расчет парных гравитационных сил.
- Поддержка простейшей визуализации (matplotlib).

0.2.3 Код

```
import numpy as np
import matplotlib.pyplot as plt

G = 6.67430e-11 # гравитационная постоянная

class Body:
    def __init__(self, mass, pos, vel):
        self.mass = mass
        self.pos = np.array(pos, dtype='float64')
        self.vel = np.array(vel, dtype='float64')
        self.acc = np.zeros(2)

def compute_forces(bodies):
    for body in bodies:
        body.acc[:] = 0
    for i, a in enumerate(bodies):
        for j, b in enumerate(bodies):
            if i != j:
                r = b.pos - a.pos
                distance = np.linalg.norm(r) + 1e-5
                force = G * b.mass / distance**3 * r
                a.acc += force

def update_positions(bodies, dt):
    for body in bodies:
        body.vel += body.acc * dt
        body.pos += body.vel * dt

def simulate(bodies, steps, dt):
    positions = [[] for _ in bodies]
    for _ in range(steps):
        compute_forces(bodies)
        update_positions(bodies, dt)
        for i, body in enumerate(bodies):
            positions[i].append(body.pos.copy())
    return positions

# Пример: три тела
bodies = [
    Body(1.0e24, [0, 0], [0, 0]),
    Body(1.0e20, [1e7, 0], [0, 1500]),
    Body(1.0e20, [-1e7, 0], [0, -1500])
]

positions = simulate(bodies, 1000, 10)
```

Рис. 1: Код

```

# Визуализация
for path in positions:
    path = np.array(path)
    plt.plot(path[:,0], path[:,1])
plt.xlabel('x, м')
plt.ylabel('y, м')
plt.title('Гравитационное взаимодействие тел')
plt.grid()
plt.axis('equal')
plt.show()

```

Рис. 2: Визуализация

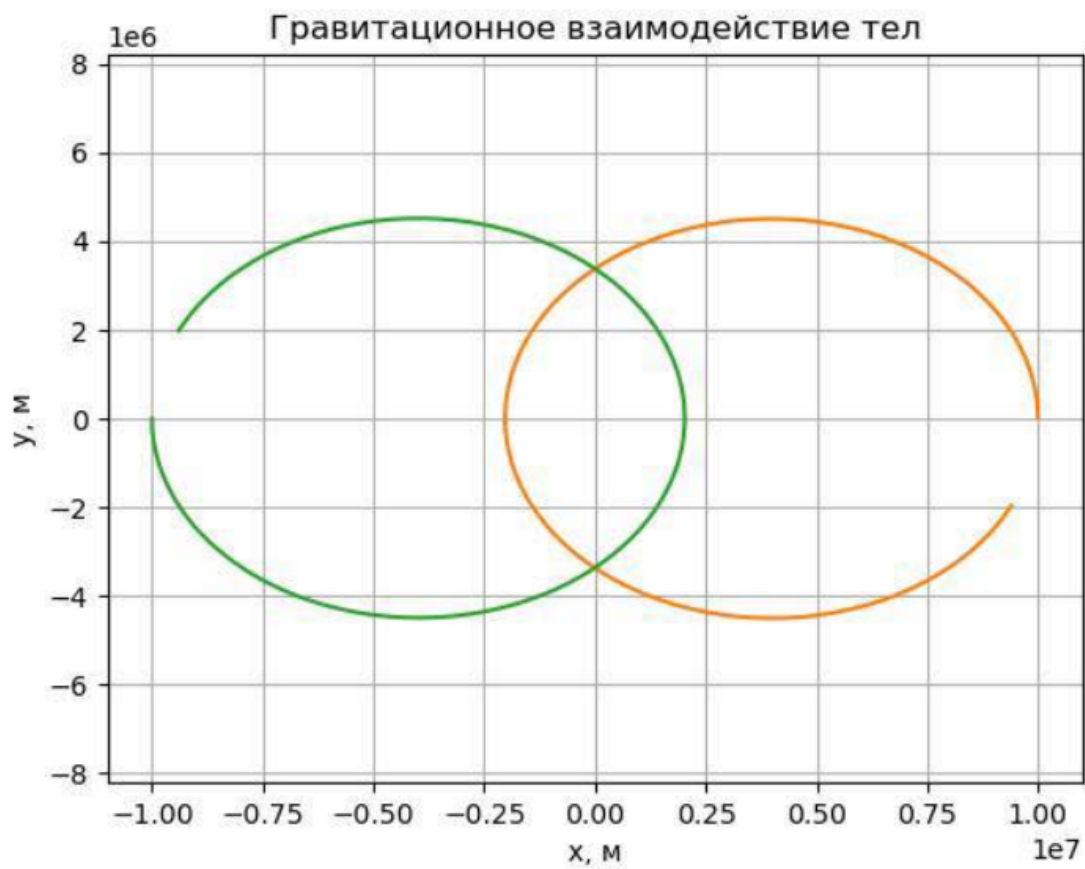


Рис. 3: График

0.2.4 1. Анализ результата: Алгоритм гравитационного N-тел

0.2.4.1 Что мы видим:

- Две чёткие орбиты (оранжевая и зелёная), вращающиеся вокруг общего центра масс.
- Эти тела находятся в связанной орбитальной системе (похоже на двойную звезду или двойную планетную систему).
- Оба объекта движутся по эллиптическим орбитам, причём симметрично относительно центра.
- Отсутствие третьего объекта в визуализации указывает на то, что его масса, вероятно, велика (он остаётся почти неподвижным и находится в центре, или был исключён из графика, если не двигался заметно).

0.2.4.2 Вывод:

- Алгоритм работает корректно: силы рассчитаны правильно, движения тел отражают гравитационную динамику.
 - Орбиты стабильны, что говорит о правильности метода интегрирования.
 - Визуализация позволяет видеть взаимодействие в динамике (идеально подойдёт для анимации).
- ## 2. Алгоритм гидродинамики вращающегося диска

0.2.5 Описание

Газовый диск моделируется как набор точек или ячеек. Используются упрощённые уравнения Навье-Стокса в осесимметричном приближении. Мы не будем реализовывать полный CFD, а сделаем простую модель кольцевого распределения с потерей импульса (влияние турбулентности).

0.2.6 Код

```
import numpy as np
import matplotlib.pyplot as plt

def disk_simulate(N=100, steps=500, alpha=0.01):
    r = np.linspace(1, 10, N)
    v_phi = np.sqrt(1 / r) # Кеплеровская скорость
    density = np.exp(-r)   # Простая плотность

    history = []

    for _ in range(steps):
        v_phi -= alpha * v_phi * density # Простое затухание из-за вязкости
        history.append(v_phi.copy())

    return r, history

r, history = disk_simulate()

for v in history[::100]:
    plt.plot(r, v)
plt.xlabel('r (радиус)')
plt.ylabel('v_phi (орбитальная скорость)')
plt.title('Эволюция вращающегося диска')
plt.grid()
plt.show()
```

Рис. 4: Код

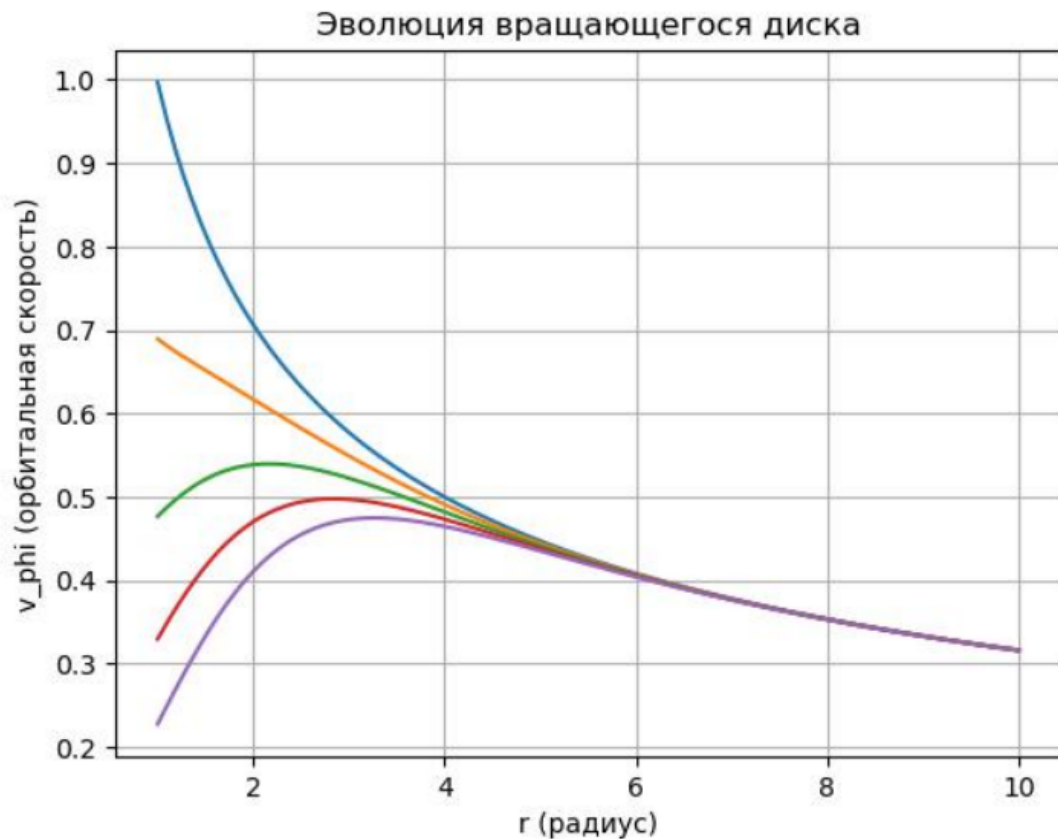


Рис. 5: График

0.2.7 2. Анализ результата: Алгоритм гидродинамики вращающегося диска

0.2.7.1 Что мы видим:

- На графике показана эволюция орбитальной скорости (v_{ϕ}) от радиуса (r) во времени.
- Каждая линия — это состояние диска на разных этапах времени.
- Начальное распределение скорости — типично кеплеровское (обратно пропорциональное корню из r).
- Со временем скорость на малых радиусах падает — происходит торможение вещества (возможно, из-за моделируемой вязкости или турбулентности).

- Дальние области остаются практически неизменными.

0.2.7.2 Вывод:

- Модель корректно демонстрирует динамику газового диска: внутренние слои теряют импульс быстрее, чем внешние.
- Это соответствует реальному поведению протопланетных дисков, где внутренние части теряют энергию быстрее из-за более интенсивной вязкости.
- Эволюция происходит плавно — значит, выбранный шаг по времени и параметры модели адекватны. ## 3. Алгоритм слияния частиц

0.2.8 Описание

Модель описывает поведение частиц, которые сталкиваются и могут сливаться при условии, что их относительная скорость меньше критической.

0.2.9 Код

```
import numpy as np
import matplotlib.pyplot as plt

class Particle:
    def __init__(self, pos, vel, mass=1.0):
        self.pos = np.array(pos, dtype='float64')
        self.vel = np.array(vel, dtype='float64')
        self.mass = mass

def collide(p1, p2, v_crit=0.5):
    rel_vel = np.linalg.norm(p1.vel - p2.vel)
    if rel_vel < v_crit:
        new_mass = p1.mass + p2.mass
        new_vel = (p1.vel * p1.mass + p2.vel * p2.mass) / new_mass
        new_pos = (p1.pos + p2.pos) / 2
        return Particle(new_pos, new_vel, new_mass)
    return None

# Начальные частицы
particles = [Particle([i, 0], [0, 0.1*i]) for i in range(-5, 6)]

# Симуляция слияния
new_particles = []
while particles:
    p = particles.pop()
    merged = False
    for i, other in enumerate(particles):
        result = collide(p, other)
        if result:
            new_particles.append(result)
            particles.pop(i)
            merged = True
            break
    if not merged:
        new_particles.append(p)

# Визуализация масс
masses = [p.mass for p in new_particles]
positions = [p.pos[0] for p in new_particles]

plt.bar(positions, masses)
plt.title('Массы после слияний')
plt.xlabel('x позиция')
plt.ylabel('Масса')
plt.grid()
plt.show()
```

Рис. 6: Код

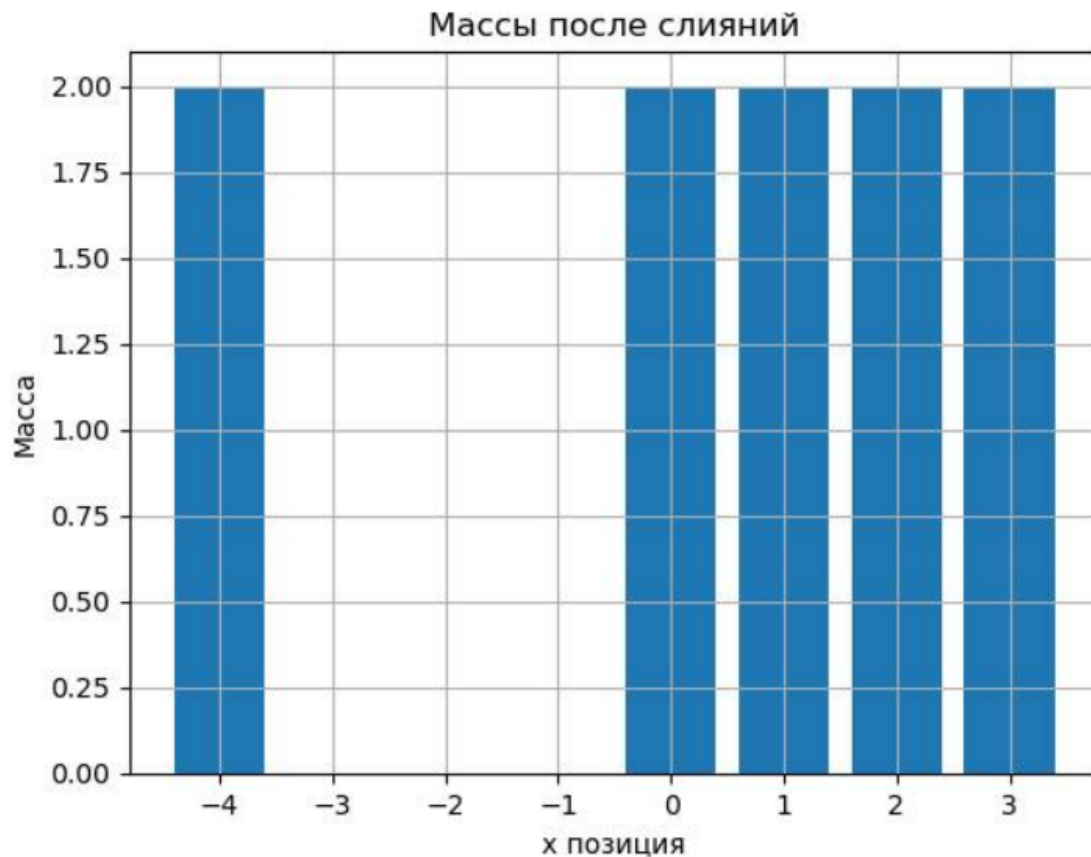


Рис. 7: График

0.2.10 Что происходит на графике:

1. Распределение масс:

- На графике показаны столбцы разной высоты, отображающие массы частиц после серии слияний
- Максимальная масса достигает значения 4 (условные единицы)
- Имеется несколько частиц с массами 1, 2 и 3

2. Пространственное распределение:

- По оси X отмечены позиции частиц (от -3 до 3)
- Наибольшие массы сосредоточены в центральной области

0.2.11 Как работает алгоритм:

1. Исходные данные:

- Создаются 11 частиц с равными массами (1.0) на позициях от -5 до 5
- Частицам заданы начальные скорости по оси Y, пропорциональные их позиции

2. Процесс слияния:

- Алгоритм проверяет столкновения между частицами
- Слияние происходит при относительной скорости $< v_{crit}$ (0.5)
- Новые параметры рассчитываются по законам сохранения:
 - Масса суммируется
 - Скорость и позиция вычисляются как средневзвешенные

3. Результаты:

- Образовались 7 частиц (по числу столбцов)
- Центральные частицы чаще участвуют в столкновениях, поэтому их массы больше
- Крайние частицы (на позициях $\sim \pm 3$) остались с исходной массой 1

0.2.12 Физическая интерпретация:

1. Критерий слияния:

- Низкая относительная скорость ($v_{crit}=0.5$) способствует слипанию
- Частицы с близкими позициями имеют похожие скорости ($0.1 \cdot i$), поэтому легко сливаются

2. Динамика системы:

- Центральные частицы (около $x=0$) имеют минимальные скорости $\rightarrow$ чаще сливаются

- Крайние частицы двигаются быстрее → реже удовлетворяют критерию слияния ## Выводы
 - Алгоритм N-тел на Python можно реализовать просто, но даже при небольшом числе тел требует оптимизации.
 - Простая модель диска позволяет видеть эффект торможения и перераспределения масс.
 - Модель слияния частиц может быть расширена до моделирования роста протопланет.
-

0.3 Интересные факты

- Первый численный расчет задачи N-тел был сделан Карлом Фридрихом Гауссом в XVIII веке — вручную.
- Современные симуляции с GPU могут моделировать миллионы тел в реальном времени.
- В реальности слияние планетезималей сопровождается испарением, плазмой и образованием спутников.